BASELINE AI WORKFORCE OS

DigitalOcean Production Deployment

Mission Control supervises · Baseline OS coordinates · Hermes / OpenClaw / Claude Code execute

Document: ops-digitalocean-execution.pdf
Source: docs/operations/DIGITALOCEAN_EXECUTION.md
Generated: 2026-05-29

> Assumes you've never deployed this before. Every step has an exact command, an exact value, and an exact verification. 45 minutes end-to-end on the happy path.

---

1. Hosting decision: App Platform — not Droplets

Use case	Recommendation	Why
Production launch	App Platform	Managed TLS, zero-config rolling deploys, built-in health checks, $12/mo entry tier. No SSH, no Caddy, no patching.
Staging copy	App Platform (separate app)	$5–12/mo. Same image, different env.
Cost-optimised self-host	Droplet (s-2vcpu-4gb, $24/mo)	Only if you need to colocate the OpenClaw gateway in the same box. Not the launch path.

Default: ship on App Platform. This doc covers App Platform only.
The droplet path is documented at the end as a backup.

App Platform sizing

Component	Tier	Monthly	Notes
`web` service	basic-xs (1× 0.5 CPU, 512 MB)	$5	Enough for ~25 concurrent operators. Hop to `basic-s` ($12) once you cross 100 concurrent.
Static health probe	Built-in (free)	$0	Polls `/api/status?action=health` every 30 s.
TLS / cert	Built-in (free)	$0	Let's Encrypt, automatic.
Outbound bandwidth	100 GB included	$0	Plenty for first 1k MAU.
Total at launch		~$5 / mo	

Don't pay for an autoscaling tier on day one — manual upgrade is one `doctl apps update` away.

---

2. Domain structure (do this before anything else)

Subdomain	Purpose	Required at launch?
`mission.baselineautomations.com`	Mission Control web app	Yes
`staging.baselineautomations.com`	Staging copy (same image)	Recommended
`docs.baselineautomations.com`	Public-facing docs / runbooks	Optional v1
`flightdeck.baselineautomations.com`	Auto-update endpoint for the desktop app	Backlog (v0.2)
`api.baselineautomations.com`	If you ever split the API	Do not split today

Buy the apex `baselineautomations.com` if you don't own it (~$12/yr, Namecheap / Porkbun). Keep the registrar separate from the host — makes future moves trivial.

Exact DNS records (Cloudflare DNS recommended; works on any provider)

After the App is created in step 5 below, DO hands you the connect target. Add these:

Type	Host	Value	Proxy	TTL
`CNAME`	`mission`	`<your-do-app>.ondigitalocean.app`	DNS-only	Auto
`CNAME`	`staging`	`<your-staging-do-app>.ondigitalocean.app`	DNS-only	Auto
`CAA`	`@`	`0 issue "letsencrypt.org"`	n/a	Auto
`MX` (if you'll send sales email from the domain)	`@`	per your provider	n/a	Auto
`TXT` (SPF)	`@`	`v=spf1 include:_spf.google.com ~all` (example)	n/a	Auto

Important: keep proxy / orange-cloud off for `mission` and `staging`. Mission Control terminates TLS at App Platform; Cloudflare in proxy mode adds a second TLS hop that breaks the WebSocket / SSE endpoints used by the activity feed.

Verification:

```
dig +short mission.baselineautomations.com
# Expect: <something>.ondigitalocean.app
```

3. Exact secrets checklist (generate before the deploy)

Run these locally (Mac / Linux):

```
openssl rand -hex 32                                          # AUTH_SECRET
openssl rand -hex 32                                          # SHARE_SIGNING_SECRET
openssl rand -hex 16                                          # API_KEY
openssl rand -base64 32 | tr -dc 'A-Za-z0-9' | head -c 32      # AUTH_PASS
```

Paste each one into your password manager before pasting into DO.

Secret	Where	Source
`AUTH_USER`	DO App env	`admin` (or your chosen operator handle)
`AUTH_PASS`	DO App env (`SECRET`)	the openssl output
`AUTH_SECRET`	DO App env (`SECRET`)	the openssl output
`API_KEY`	DO App env (`SECRET`)	the openssl output
`SHARE_SIGNING_SECRET`	DO App env (`SECRET`)	the openssl output
`STRIPE_SECRET_KEY`	DO App env (`SECRET`)	Stripe Dashboard → Developers → API keys → "Reveal live key" (or `sk_test_...` to launch in mock mode)
`STRIPE_WEBHOOK_SECRET`	DO App env (`SECRET`)	Stripe Dashboard → Webhooks → after creating the endpoint in step 8 below
`DIGITALOCEAN_ACCESS_TOKEN`	GitHub repo Secrets	DO → API → Personal Access Tokens, full scope
`DIGITALOCEAN_APP_ID`	GitHub repo Secrets	captured after `doctl apps create` in step 5
`MC_HOST`	GitHub repo Secrets + DO env	`mission.baselineautomations.com`
`MC_DOMAIN_ZONE`	DO env	`baselineautomations.com`

Plain (non-secret) DO env — already templated in `.do/app.yaml`:

Variable	Value
`NODE_ENV`	`production`
`PORT`	`3000`
`MC_COOKIE_SECURE`	`1`
`MC_COOKIE_SAMESITE`	`strict`
`MC_ENABLE_HSTS`	`1`
`MC_ALLOWED_HOSTS`	`mission.baselineautomations.com`
`OPENCLAW_GATEWAY_HOST`	`127.0.0.1`
`OPENCLAW_GATEWAY_PORT`	`18789`

Now run the local preflight to catch typos before uploading:

```
./scripts/preflight-production.sh .env.production
# Expect: "Preflight PASSED" (warnings about Stripe mock-mode are OK)
```

---

4. Install the DO CLI

```
# Mac
brew install doctl

# Linux
sudo snap install doctl

# Windows
choco install doctl
```

Authenticate:

```
doctl auth init
# Paste the DIGITALOCEAN_ACCESS_TOKEN you generated above
doctl account get      # Should print your account email
```

---

5. Exact deployment sequence

```
# A. From the repo root, on the deploy SHA
git status                                          # clean working tree
git log -1 --oneline                                # capture this SHA in the launch ticket

# B. Push the image. CI builds it via .github/workflows/docker-publish.yml
git push origin main
#   → GitHub Actions: Quality Gate (vitest 1214/1214) → Docker Publish → GHCR.

# C. Wait for the image. Watch:
gh run watch --exit-status                          # if you have gh CLI

# D. Create the App from the spec (FIRST DEPLOY ONLY)
doctl apps create --spec .do/app.yaml
#   → outputs the new App ID. Save it.

APP_ID=<paste app id>
echo "$APP_ID" >> ~/.baseline-app-id                # for next time

# E. Stamp the DO secrets (one-time)
for var in AUTH_USER AUTH_PASS AUTH_SECRET API_KEY SHARE_SIGNING_SECRET \
           MC_ALLOWED_HOSTS STRIPE_SECRET_KEY; do
  doctl apps spec get $APP_ID > /tmp/spec.yaml
  # Edit /tmp/spec.yaml — replace the SECRET stubs with your real values,
  # OR use the DO Dashboard → Settings → Environment Variables.
done
doctl apps update $APP_ID --spec /tmp/spec.yaml --wait

# F. Add the GitHub repo Secrets:
#     DIGITALOCEAN_ACCESS_TOKEN, DIGITALOCEAN_APP_ID, MC_HOST
#     Settings → Secrets → Actions → New repo secret

# G. Attach the custom domain (in the DO Dashboard or via spec)
#     Apps → <app> → Settings → Domains → Add mission.baselineautomations.com
#     DO will give you the CNAME target. Use it in step 2's DNS table.

# H. Watch logs through the first deploy
doctl apps logs $APP_ID --follow --type=run

# I. Subsequent deploys are automatic on every push to main
#     (.github/workflows/deploy-digitalocean.yml takes over).
```

Acceptance: the App Platform dashboard shows the deployment phase as `ACTIVE` and the health probe is green. If either is red, jump to section 7 (rollback) below.

---

6. Post-deploy verification

Hand off to PRODUCTION_VERIFICATION_CHECKLIST.md in this folder. That's the only acceptance gate. Don't post the launch link until every checkbox is green.

---

7. Exact rollback sequence

The deploy workflow auto-rolls back if `/api/status?action=health` isn't `healthy` within ~150 s. If you need to do it manually:

A. Re-pin the previous good image tag

```
doctl apps list-deployments $APP_ID
#  ID                                   PHASE      CREATED AT            CAUSE
#  abc1234-...                          ACTIVE     2026-05-29T09:00:00Z  user
#  def5678-...                          SUCCESS    2026-05-28T18:00:00Z  push to main   ← previous good

# Get the image tag the previous deploy used (it's the commit SHA)
doctl apps get-deployment $APP_ID def5678-... --format Spec.Services.0.Image.Tag

# Re-deploy with that tag
doctl apps update $APP_ID --spec .do/app.yaml --image-tag sha-<previous-sha>
```

B. Revert the bad commit

```
git revert <bad-sha>
git push origin main
#   → CI rebuilds the image; deploy workflow auto-deploys the revert.
```

C. Rotate the share-signing secret (if it leaked)

```
NEW_SECRET=$(openssl rand -hex 32)
# DO Dashboard → Settings → Env vars → SHARE_SIGNING_SECRET = $NEW_SECRET
# Save. DO redeploys. Every live demo link instantly hits /demo/expired.
```

D. Full freeze (extreme)

```
# Set the instance count to 0 in .do/app.yaml, push, deploy.
# Visitors get a DO maintenance page until you scale back.
```

Detail: `docs/operations/ROLLBACK.md`.

8. Stripe (only when ready to charge real money)

1. Stripe Dashboard → Developers → Webhooks → Add endpoint:
   `https://mission.baselineautomations.com/api/stripe/webhook`
2. Subscribe events:
   `checkout.session.completed`,
   `checkout.session.async_payment_succeeded`
3. Copy the signing secret to DO env as `STRIPE_WEBHOOK_SECRET`.
   Save → DO redeploys automatically.
4. Test with a $1 token-pack purchase end-to-end. Confirm the ledger credits inside Mission Control's Billing panel.
5. Send the webhook again from the Stripe Dashboard ("Send test webhook") and confirm no double credit (idempotency).

Until step 5 passes, leave the production env on `sk_test_...` and mock-mode billing.

9. Droplet path (only if you must self-host)

Resource	Size	Cost	Notes
Droplet	`s-2vcpu-4gb` (Ubuntu 22.04 LTS)	$24/mo	Tightly sized for v1
Block storage	25 GB volume	$2.50/mo	SQLite + backups
Reserved IP	included	$0	Stable inbound IP

Setup (high level):

```
# 1. SSH in
ssh root@<droplet-ip>

# 2. Pull the image
docker login ghcr.io
docker pull ghcr.io/builderz-labs/mission-control:latest

# 3. Run with the hardened Dockerfile + Caddy in front
docker compose -f /app/docker-compose.production.yml up -d
```

Use the included `Caddyfile.production` for TLS + reverse proxy.
The same `.env.production` from section 3 applies. Cron the SQLite backup from `docs/operations/BACKUP_RESTORE.md`.

This path is unsupported on day one — only choose it if you need to colocate runtime sidecars and you're willing to own the OS.

Quick reference

Goal	Command
First deploy	`doctl apps create --spec .do/app.yaml`
Re-deploy	`git push origin main`
Watch build	`doctl apps logs $APP_ID --follow --type=build`
Watch runtime	`doctl apps logs $APP_ID --follow --type=run`
Check health	`curl https://mission.baselineautomations.com/api/status?action=health`
List deploys	`doctl apps list-deployments $APP_ID`
Roll forward	`git push origin main`
Roll back	`doctl apps update $APP_ID --image-tag sha-<prev>`
Rotate share secret	DO env → `SHARE_SIGNING_SECRET`
Scale up	DO env → `instance_size_slug` = `basic-s`